# Chapter 2, part 1

## Systems of linear equations

- intro systems of linear equations
- LU decomposition (without pivoting, i.e. row swaps)
- algorithm and cost
- vector and matrix norms
- condition number

# Recall matrix-vector product

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n \end{bmatrix} =$$

matrix $A$ — vector $x$

$$= x_1 \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{bmatrix} + x_2 \begin{bmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{n2} \end{bmatrix} + \cdots + x_n \begin{bmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{nn} \end{bmatrix}$$

columns of $A$

# Systems of linear equations

Example

$$\left.\begin{aligned} 4x_1 + 3x_2 &= 5 \\ 2x_1 - x_2 &= -1 \end{aligned}\right\}$$

Matrix form

$$\begin{bmatrix} 4x_1 + 3x_2 \\ 2x_1 - x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \end{bmatrix}$$

$$\underbrace{\begin{bmatrix} 4 & 3 \\ 2 & -1 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 5 \\ -1 \end{bmatrix}}_{b}$$

In general, solve $x$ from

$$Ax = b$$

$A$ $n \times n$ matrix

$x, b$ are vectors length $n$

# Matrix inverse $A^{-1}$

Let $A \in \mathbb{R}^{n \times n}$. The matrix inverse $A^{-1}$, if it exists, satisfies

$$A^{-1}A = AA^{-1} = I$$

Equivalent are:

1. $A^{-1}$ exists
2. $\det(A) \neq 0$
3. $\operatorname{rank}(A) = n$
4. $Ax \neq 0$ whenever $x \in \mathbb{R}^n, x \neq 0$.

If 1. - 4. hold $A$ is called nonsingular and $Ax = b$ has a unique solution $x = A^{-1}b$

Remarks

- $A^{-1}$ used for the theory
- in practice $Ax = b$ is solved using the LU decomposition

## No $A^{-1}$, no unique solutions

If $A^{-1}$ does not exist (1.-4. do not hold then $A$ is called singular.

Two possibilities

(A) $Ax = b$ has no solutions

(B) $Ax = b$ has infinitely many solutions.

Example: $A = \begin{bmatrix} 2 & 0 & 1 \\ 1 & -1 & 3 \\ 3 & -1 & 4 \end{bmatrix}$ $b = \begin{bmatrix} 2 \\ 0 \\ 2 \end{bmatrix}$

Apply Gaussian elimination

$$\left[\begin{array}{ccc|c} 2 & 0 & 1 & 2 \\ 1 & -1 & 3 & 0 \\ 3 & -1 & 4 & 2 \end{array}\right] \begin{array}{l} \\ \leftarrow \text{subtract } \frac{1}{2} \text{ row } 1 \\ \leftarrow \text{subtract } \frac{3}{2} \text{ row } 1 \end{array}$$

$$\sim \left[\begin{array}{ccc|c} 2 & 0 & 1 & 2 \\ 0 & -1 & 2\frac{1}{2} & -1 \\ 0 & -1 & 2\frac{1}{2} & -1 \end{array}\right] \sim \left[\begin{array}{ccc|c} 2 & 0 & 1 & 2 \\ 0 & -1 & 2\frac{1}{2} & -1 \\ 0 & 0 & 0 & 0 \end{array}\right]$$

$$\left.\begin{array}{l} 2x_1 + x_3 = 2 \\ -x_2 + 2\frac{1}{2}x_3 = -1 \\ 0 = 0 \end{array}\right\} \quad \begin{array}{l} x_1 = \frac{1}{2}(2 - x_3) \\ x_2 = -(-1 - 2\frac{1}{2}x_3) \end{array}$$

$\infty$ many solutions

rank $A = 2$

# LU decomposition (without pivoting)

# LU decomposition

Recall matrix properties:

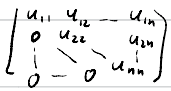
$U$ is **upper triangular** if

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}$$

So $u_{ij} = 0$ if $i > j$

$L$ is **lower triangular** if

$$L = \begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & 0 \\ l_{n1} & \cdots & \cdots & l_{nn} \end{bmatrix}$$



$l_{ij} = 0$ if $i < j$

In LU decomposition we write

$$A = LU$$

$L$: lower triangular

$U$: upper triangular

# Example LU decom

Idea: transform $A$ to $U$ by creating zeros below diagonal

$$A = \begin{bmatrix} 2 & 4 & 3 & 1 \\ 4 & 9 & 7 & 4 \\ -2 & -8 & -10 & -7 \\ 4 & 9 & -2 & 6 \end{bmatrix}$$

create zero here:

subtract $\boxed{2}$ · row 1 from row 2

subtract $\boxed{-1}$ · row 1 from row 3

subtract $\boxed{2}$ · row 1 from row 4

↑ these go there

$$A_1 = \text{"new } A\text{"} = \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & -4 & -7 & -6 \\ 0 & 1 & -8 & 4 \end{bmatrix}$$

Next: Write $A = L_1 A_1$

first step of $L$ ↗

$$A = \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & & 1 & \\ 2 & & & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & -4 & -7 & -6 \\ 0 & 1 & -8 & 4 \end{bmatrix}$$

$$A = L_1 A_1 = \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & & 1 & \\ 2 & & & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & -4 & -7 & -6 \\ 0 & 1 & -8 & 4 \end{bmatrix}$$

subtract $-4 \cdot$ row 2 from row 3

subtract $1 \cdot$ row 2 from row 4

$$= \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & & 1 & \\ 2 & & & 1 \end{bmatrix} \begin{bmatrix} 1 & & & \\ & 1 & & \\ & -4 & 1 & \\ & 1 & & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & -3 & 2 \\ 0 & 0 & -9 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & -4 & 1 & \\ 2 & 1 & & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & -3 & 2 \\ 0 & 0 & -9 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & -4 & 1 & \\ 2 & 1 & & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & -3 & 2 \\ 0 & 0 & -9 & 2 \end{bmatrix}$$

subtract 3 · row 3 from row 4

$$= \begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & -4 & 1 & \\ 2 & 1 & & 1 \end{bmatrix} \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & -3 & 2 \\ 0 & 0 & 0 & -4 \end{bmatrix}$$

$$= \underbrace{\begin{bmatrix} 1 & & & \\ 2 & 1 & & \\ -1 & -4 & 1 & \\ 2 & 1 & 3 & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} 2 & 4 & 3 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & -3 & 2 \\ 0 & 0 & 0 & -4 \end{bmatrix}}_{U}$$

$\Rightarrow$ LU decom found

# A triangular solve

Let's solve $Lx = b$

$$L = \begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & 0 & \cdots \\ \vdots & \vdots & \ddots & 0 \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

First line:

$$l_{11} x_1 = b_1 \Rightarrow x_1 = \frac{b_1}{l_{11}}$$

2nd line:

$$l_{21} x_1 + l_{22} x_2 = b_2 \Rightarrow$$

$$l_{22} x_2 = b_2 - l_{21} x_1$$

$$x_2 = \frac{1}{l_{22}} (b_2 - l_{21} x_1)$$

k-th line (works for $k = 1 \dots n$)

$$l_{k,1} x_1 + l_{k,2} x_2 + \dots + l_{k,k-1} x_{k-1} + l_{k,k} x_k = b_k$$

(known, solved previously)

$$x_k = \frac{1}{l_{k,k}} \left( b_k - l_{k,1} x_1 - l_{k,2} x_2 - \dots - l_{k,k-1} x_{k-1} \right)$$

→ called forward substitution

→ solving $Lx = b$ is easy

requirement that $l_{k,k} \neq 0$

$k = 1 \dots n$.

# Solving $Ax = b$ using $A = LU$

$LUx = b$

$\underbrace{Ux}_{y}$

$Ly = b \Rightarrow$ can be solved
$\Rightarrow$ know $y$

$Ux = y \Rightarrow$ can be solved
$\Rightarrow$ know $x$

Solving $Ax = b$ using $A = LU$

1. Solve $y$ from $Ly = b$
2. Solve $x$ from $Ux = y$.

Note: $Ux = y$ solved like $Ly = b$, but bottom up; called backward substitution

## Recap solving $Ax = b$

- Compute $L$ (lower triangular) $U$ upper triangular s.t.
$$A = LU$$

- Solve $Ly = b$
$Ux = y$
using algorithms for triangular systems of eqs. called forward substitution and backward substitution

## Remarks

- There is division $(a_{jj})^{-1}$
$\Rightarrow$ must have $a_{jj} \neq 0$
otherwise row swaps (next lecture)

- $LU$ decom with row swaps standard method for solving $Ax = b$
Hardly ever compute $A^{-1}$!

- If solving $Ax = b$ for multiple $b$, compute $(L, U)$ only once.

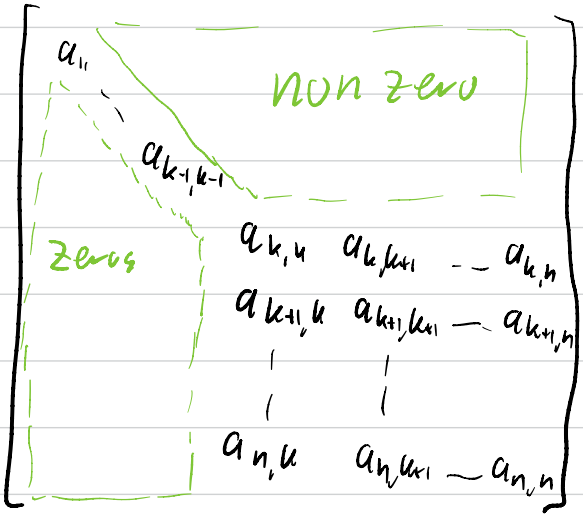
# Algorithm and cost

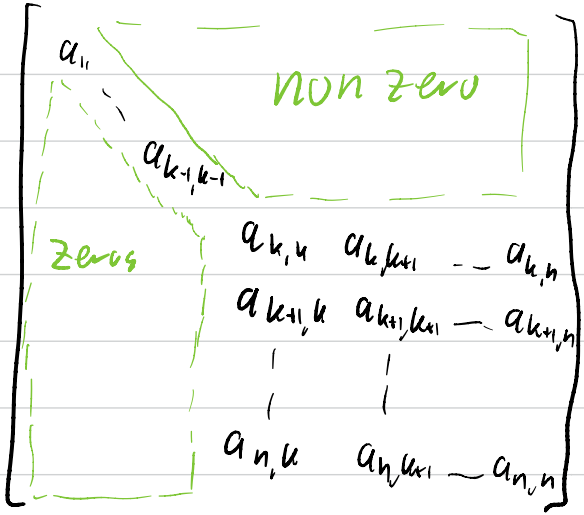
# Algorithm

- Algorithm: Steps the computer must take in pseudocode

- Analyze the creation of zeros in column k.

$A_{k-1} =$






Step $k$:

- compute $l_{j,k} = \frac{a_{j,k}}{a_{k,k}}$ $j = k+1 \dots n$
- subtract $l_{j,k} \cdot a_{k,j}$ from $a_{i,j}$

loop $k = n, n-1, \dots, 2$

## Algorithm (Heath):

**for** $k = 1$ **to** $n-1$
  **if** $a_{kk} = 0$ **then** stop
  **for** $i = k+1$ **to** $n$
    $m_{ik} = a_{ik}/a_{kk}$
  **end**
  **for** $j = k+1$ **to** $n$
    **for** $i = k+1$ **to** $n$
      $a_{ij} = a_{ij} - m_{ik}a_{kj}$
    **end**
  **end**
**end**

# Cost of LU decomposition

Heath defines cost of algorithm by

cost = # additions or # multiplications + # divisions in algorithm

To compute cost:
- write down algorithm
- look at loops and operations, add up # operations for each time you go through loop

```
1.  for k = 1 to n − 1
2.      if a_kk = 0 then stop
3.      for i = k + 1 to n
4.          m_ik = a_ik / a_kk
7.      end
6.      for j = k + 1 to n
7.          for i = k + 1 to n
8.              a_ij = a_ij − m_ik a_kj
9.          end
10.     end
11. end
```

cost $n-k$

cost $n-k$

cost $(n-k)(n-k)$

**Cost:**

$$\sum_{k=1}^{n-1} \left((n-k)^2 + n-k\right)$$

$$= (n-1)^2 + (n-2)^2 + \cdots + 1$$

$$(n-1) + (n-2) + \cdots + 1$$

$$= \frac{1}{6}(n-1)n(2n-1) + \frac{1}{2}(n-1)n$$

$$= \frac{1}{3}n^3 + \underline{O(n^2)}$$

negligible compared to $\frac{1}{3}n^3$
standard omitted

---

sum formulas:

$$1+2+\cdots+n = \frac{1}{2}n(n+1)$$

$$1+2^2+3^2+\cdots+n^2 = \frac{1}{6}n(n+1)(2n+1)$$

Validation (puzzle).

# Matrix and vector norms

# Vector norms

You know the 2-norm of a vector $x$

$$\|x\|_2 = \sqrt{x_1^2 + \dots + x_n^2}$$

measures the size of $x$.

We need this for error estimates:

if $x$ = true solution

$\hat{x}$ = approx solution we want

size of $\hat{x} - x$ is small

can be 2-norm or other norm

Two other norms are

$\|x\|_1 = |x_1| + \dots + |x_n|$

1-norm (or Manhattan norm)

$\|x\|_\infty = \max\{|x_1|, |x_2|, \dots, |x_n|\}$

sup norm

# Remarks about norms

① A **norm** is a function $x \mapsto \|x\|$ s.t.

(i) $\|x\| \geq 0$ and $\|x\| = 0 \iff x = 0$

(ii) $\|\lambda x\| = |\lambda| \, \|x\|$

(iii) $\|x+y\| \leq \|x\| + \|y\|$ (triangle inequality)

② Norms on $\mathbb{R}^n$ differ by at most a constant factor.

Norm equivalence

$\|\cdot\|_1$ and $\|\cdot\|_2$ equivalent $\iff$ there are $C_1, C_2 > 0$ s.t.

$C_1 \|x\|_1 \leq \|x\|_2 \leq C_2 \|x\|_1$ for all $x \in \mathbb{R}^n$

E.g. $\frac{1}{\sqrt{n}} \|x\|_1 \leq \|x\|_2 \leq \|x\|_1$

$\frac{1}{\sqrt{n}} \|x\|_\infty \leq \|x\|_\infty \leq \|x\|_2$

$\frac{1}{n} \|x\|_1 \leq \|x\|_\infty \leq \|x\|_1$

May choose a convenient norm in given context.

# Matrix norms

We also need matrix norms

- for defining condition number

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

- for estimating effect of errors in $A$

$$\|\hat{x} - x\| \leq (\text{some number}) \cdot \|\hat{A} - A\|$$

(not really discussed)

# Matrix norms

Let $\|\cdot\|$ be a vector norm and $A \in \mathbb{R}^{n \times n}$. We define

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

Defined for different vector norms, e.g.

$$\|A\|_1 = \max_{x \neq 0} \frac{\|Ax\|_1}{\|x\|_1}$$

For all $x \in \mathbb{R}^n$

$$\frac{\|Ax\|}{\|x\|} \leq \|A\|$$

so

$$\|Ax\| \leq \|A\| \, \|x\|$$

$\Rightarrow \|A\|$ is "max multiplication factor" of $\|x\|$.

Also

$$\|A \cdot B\| \leq \|A\| \cdot \|B\|$$

$\|A\|$ satisfies norm properties

# Computing matrix norms

One can show that

$$\|A\|_1 = \max_j \sum_{i=1}^{n} |a_{ij}| \longrightarrow \text{sum elts of columns}$$

$$\|A\|_\infty = \max_i \sum_{j=1}^{n} |a_{ij}| \longrightarrow \text{sum elts of raw}$$

Computation of $\|A\|_2$ in Chap 3.

# Condition number

Suppose we solve $Ax = b$

$b$ = true input $\quad x$ = true output

$\hat{x}$ = approx output

$\hat{b}$ such that $A\hat{x} = \hat{b}$

(input s.t. $\hat{x}$ is the output)

Forward error = $\hat{x} - x$

backward error = $\hat{b} - b$.

we will derive formula for

$$\text{cond} = \max_{\text{over possible inputs/outputs}} \frac{\|\Delta x\| / \|x\|}{\|\Delta b\| / \|b\|}$$

## Derivation

$$\text{cond} = \max \frac{\|\Delta x\| / \|x\|}{\|\Delta b\| / \|b\|}$$

$$= \max \frac{\|\Delta x\|}{\|\Delta b\|} \cdot \frac{\|b\|}{\|x\|}$$

$$= \max \frac{\|\Delta x\|}{\|\Delta b\|} \cdot \max \frac{\|b\|}{\|x\|}$$

since $Ax = b$, $A\vec{x} = \vec{b}$

we have

$$A\Delta x = A(\vec{x} - x) = \vec{b} - b = \Delta b$$

$$\Delta x = A^{-1} \Delta b$$

$$\max \frac{\|b\|}{\|x\|} = \max \frac{\|Ax\|}{\|x\|} = \|A\|$$

$$\max \frac{\|\Delta x\|}{\|\Delta b\|} = \max \frac{\|A^{-1}\Delta b\|}{\|\Delta b\|} = \|A^{-1}\|$$

So

$$\text{cond} = \text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

# Large condition number

Easy example

$$A = \begin{bmatrix} 10 & \\ & 10^{-3} \end{bmatrix} \Rightarrow \|A\| = 10$$

$$A^{-1} = \begin{bmatrix} 10^{-1} & \\ & 10^{3} \end{bmatrix} \Rightarrow \|A^{-1}\| = 10^{3}$$

$$\text{Cond}(A) = 10^{4}$$

---

$$\text{cond}\begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{bmatrix} = \frac{\max_n |\lambda_n|}{\min_n |\lambda_n|}$$

cond(A) is also large if columns of A are nearly linear dependent

e.g. $A = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$

cf. slides Health.

# Example

Consider Hilbert matrices $H_n$

$$H_n = \begin{bmatrix} 1 & 1/2 & 1/3 & \cdots & 1/n \\ 1/2 & 1/3 & 1/4 & \cdots & 1/n+1 \\ 1/3 & 1/4 & 1/5 & \cdots & 1/n+2 \\ \vdots & & & & \\ 1/n & \cdots & \cdots & \cdots & 1/2n-1 \end{bmatrix}$$

We do the following for $n = 2, 4, 6, \ldots, 20$

```
x  = randn(n)      <= "true x"
b  = H_n * x
x̂  = solve(H_n, b)   <= "approx x"
print( ||x̂ - x|| / ||x|| )
```

Result is

| n | relErr |
|---|---|
| 2 | 5.3e-16 |
| 4 | 1.83e-13 |
| 6 | 2.47e-11 |
| 8 | 4.56e-08 |
| 10 | 3.07e-05 |
| 12 | 0.0517 |
| 14 | 0.793 |
| 16 | 36.9 |
| 18 | 0.407 |

Next: - explain this
- comes from blowing up input errors. Unavoidable for this matrix.

Example

| n | relErr | cond(Hn) | relErr / (cond · $\varepsilon_{mach}$) |
|---|---|---|---|
| 2 | 5.3e-16 | 19.3 | 0.275 |
| 4 | 1.83e-13 | 1.55e+04 | 0.118 |
| 6 | 2.47e-11 | 1.5e+07 | 0.0165 |
| 8 | 4.56e-08 | 1.53e+10 | 0.0299 |
| 10 | 3.07e-05 | 1.6e+13 | 0.0192 |
| 12 | 0.0517 | 1.75e+16 | 0.0295 |
| 14 | 0.793 | 6.2e+17 | 0.0128 |
| 16 | 36.9 | 7.87e+17 | 0.469 |
| 18 | 0.407 | 2.24e+18 | 0.00181 |

$1.9 \cdot 10^{9}$
$2.0 \cdot 10^{22}$
$2.2 \cdot 10^{25}$

relErr is roughly of size cond $(H_n) \cdot \varepsilon_{mach}$

# Conclusion

- condition number of $A$ equals

$$\text{cond}(A) = \|A^{-1}\| \cdot \|A\|$$

- large condition number means input errors (e.g. rounding errors) are **blown up**

$$\|\text{output errors}\| = \text{cond}(A)\, \|\text{input errors}\|$$